7-5 · Hybrid Search 구현 · 정리·회고 0.5H

정리·회고

찾은 근거 안에서만 답하고, 출처를 보여 준다 — 7주차

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

30분 뒤에는

① 7-1~7-5 에서 한 일과 남은 결과물을 한 표로 본다

② 체크리스트로 오늘 실습을 스스로 점검한다

③ 자주 겪는 오류 네 가지의 원인과 해결을 안다

④ docker compose version 을 확인해 8주차 Airflow 를 준비한다

7-1 의 「지어낸다」에 대한 답 — 근거 안에서만, 출처와 함께 · 작은 모델은 그래도 지어낼 수 있다

이번 주에 만든 것

닷새 — 한 일과 남은 결과물

날	한 일	남은 결과물
7-1	LLM API · 프롬프트 · 멀티턴	챗봇 골격
7-2	문서 로드 · Chunking	조각 기준값
7-3	임베딩 · Qdrant 검색	lecture_notes 컬렉션
7-4	MinIO → Qdrant 파이프라인	lecture_kb 컬렉션
7-5	BM25 + RRF + 답변 생성	RAG 챗봇 · 산출물 2종

7-1 에서 본 문제의 답 — 찾은 근거 안에서만 답하게 하고 출처를 함께 보여 준다

오늘의 요약

하려는 일 → 코드 한 줄

하려는 일	코드
전량 내려받기	client.scroll(name, limit=, with_payload=True)
BM25 색인	BM25Okapi([doc.split() for doc in corpus])
BM25 점수	bm25.get_scores(query.split())
벡터 순위	client.query_points(...).points → str(h.id)
RRF 결합	Σ 1/(k + 순위), k=60
프롬프트 체인	prompt | llm | StrOutputParser()
컨텍스트 주입	chain.invoke({"context": ..., "question": ...})

오늘의 체크리스트

여덟 줄로 스스로 확인해 보세요

1. BM25 가 무엇을 보고 점수를 매기는지 설명할 수 있다
2. 질문 유형별로 BM25 와 벡터의 강약이 갈리는 것을 실측했다
3. 한국어 조사 때문에 BM25 가 0점이 되는 것을 재현했다
4. RRF 가 점수 대신 순위를 쓰는 이유를 설명하고 손 계산으로 검증했다
5. 두 순위의 id 타입을 str 로 통일해야 하는 이유를 안다
6. k 값을 바꿔 결합 순위가 달라지는 것을 봤다
7. 컨텍스트 밖 질문의 답을 살폈고(작은 모델은 지어냄) 출처를 붙였다
8. 산출물 2종을 검증했다 — size 대조 · 두 번 실행 · grep "sk-"

자주 겪는 오류

증상으로 찾는 원인과 해결

증상	원인	해결
RRF 결과가 두 배 · 같은 문서가 안 합쳐짐	BM25 는 str, Qdrant 는 int·UUID — 타입 불일치	두 순위 모두 str(id) 로 통일
한국어 질문에 BM25 점수 전부 0	공백 토큰화 — 「도구를」≠「도구」	명사 위주 짧은 질문 · 실무는 Kiwi 로 양쪽 같게
결과 0건 · scroll 문서 수 0	컬렉션 이름 오타 · 다른 모델로 다른 컬렉션	get_collections() 대조 · emb 모델·차원 확인
새 문서가 BM25 에 안 잡힘	BM25 는 메모리 색인 — 자동 갱신 없음	scroll 다시 → BM25Okapi 새로

BM25 가 0점만 내놓으면 가장 먼저 볼 곳 — 색인과 검색의 토큰화가 같은가

예상 질문

자주 나오는 질문 네 가지

1. **Qdrant 도 키워드 검색을 지원하지 않나요?**
   지원한다(sparse vector) — 오늘은 원리를 손으로 만들어 보는 것이 목적

2. **검색기를 셋 이상 합칠 수 있나요?**
   rrf() 에 순위 목록을 하나 더 넣으면 그만 — 그것이 RRF 의 장점

3. **답변이 여전히 틀리면?**
   검색 결과부터 — 근거가 틀렸는지, 근거는 맞는데 모델이 잘못 읽었는지 나눠 본다

4. **문서가 수십만 건이면?**
   메모리 BM25 는 못 올린다 — Elasticsearch·OpenSearch 나 Qdrant 의 sparse vector

8주차를 위한 준비

Airflow 가 이 모든 것을 자동화한다

1. **docker --version · docker compose version**
   compose version 이 나오면 준비된 것(실측 v5.3.0)

2. **docker system df**
   메모리를 넉넉히 — Docker Desktop → Settings → Resources, 8GB 권장

3. **docker stop qdrant rag-minio ollama (선택)**
   이번 주 컨테이너는 두어도 되고, 자원이 부족하면 멈춰 둔다

8주차 재료

build_kb.py

→ Airflow DAG

다음 회차 예고

8-1 · Airflow 서비스 구축

01
이론 1H
오케스트레이션과
Airflow 개요

02
실습 1.5H
Compose 로
Airflow 기동

03
실습 1H
Airflow 웹 UI ·
CLI 탐색

04
정리 0.5H
구성 요소 복습 ·
환경 상태 점검

준비: Docker Desktop 실행 · 할당 메모리 최소 4GB(권장 8GB) · Spark·Qdrant 등 이전 주차 컨테이너는 꺼 두어도 됨

7-5 · 정리·회고 0.5H 마무리

다음 회차는 8-1 Airflow 서비스 구축

손으로 실행해 온 수집·적재·변환·임베딩을 알아서 돌게 — 지휘자 Airflow 를 직접 띄우고 웹 UI 와 CLI 로 들여다봅니다

- ✓ **cron·수동 실행의 한계와 오케스트레이션**
- ✓ **웹서버 · 스케줄러 · 워커 · 메타데이터 DB**
- ✓ **공식 docker-compose 로 초기화 · 기동**
- ✓ **DAG 목록 · Grid 화면 · CLI 기본 명령**

준비할 것

Docker 메모리 8GB 권장 · 컨테이너 정리